BÖLÜM 2 / MODULE 2

Üç Okuyucu Modeli & Yapay Zekâ Çağının 5 Temel İlkesi

The Three Readers & The 5 AI-Era Principles

Use case'in üç farklı kitlesi: Paydaş (Stakeholder), Yazılım Mühendisi ve Yapay Zekâ Ajanı. AI'ın lafı harfi harfine anlaması ve boşluk bırakamaması. Use-Case 3.0'ın 10 ilkesine eklenen 5 yeni AI çağı ilkesi (İlke 11-15).

The three readers of a use case: Stakeholder, Software Engineer, and AI Agent. How AI reads literally and cannot leave gaps unfilled. The five new principles for the AI era (Principles 11-15) extending Use-Case 3.0.

7

BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

Üç Okuyucu Modeli: Paydaş Doğrular, Mühendis İnceler, AI Üretir

The Three Readers: Stakeholder Validates, Engineer Reviews, AI Generates

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
                USE CASE'İN ÜÇ OKUYUCUSU VE OKUMA ŞEKİLLERİ

OKUYUCU 1: PAYDAŞ (STAKEHOLDER)
- Ne İçin Okur? "İstediğim şey bu mu? Teslim edildiğinde anlayabilecek miyim?"
- Nasıl Okur?   Hızlıca göz gezdirir (Skim). Hikayeye, isimlere, mutlu yola bakar. Çabuk sıkılır!

OKUYUCU 2: YAZILIM MÜHENDİSİ (SOFTWARE ENGINEER)
- Ne İçin Okur? "Bu inşa edilebilir mi? Üretilen kod ve testler bu şartnameye uyuyor mu?"
- Nasıl Okur?   Satır satır inceler (Review). Akışları kod ve testlerle çapraz doğrular.

OKUYUCU 3: YAPAY ZEKÂ AJANI (AI AGENT)
- Ne İçin Okur? "Ben tam olarak ne üreteceğim?"
- Nasıl Okur?   Harfi harfine ve eksiksizce (Literally & Exhaustively). Asla sıkılmaz,
                tüm referansları takip eder ve 'ne demek istediniz' diye ASLA sormaz!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoran düşünün: 1) Müşteri (Paydaş) menüye bakar, resimleri ve yemek isimlerini hızlıca süzer, lezzetli görünüyorsa 'Bunu istiyorum' der. 2) Restoran Müdürü (Mühendis) menüye bakar; 'Mutfakta malzeme var mı, maliyet kurtarır mı, hijyen kurallarına uygun mu?' diye kontrol eder. 3) Mutfaktaki Robot Aşçı (AI) ise tarifi harfi harfine okur: '1 gram tuz, 200 derece fırın'. Robot aşçı müşterinin gözünün içine bakıp 'Tuzlu mu seversin?' diye sormaz; tarifte ne yazıyorsa milimetrik olarak tabağa koyar!

🇬🇧 ENGLISH (ELI5)

Consider a restaurant menu: 1) The Customer (Stakeholder) skims the pictures and dish names. If it looks appetizing, they order. They have zero patience for ingredient chemical formulas. 2) The General Manager (Engineer) reviews the menu to confirm inventory feasibility and safety regulations. 3) The Robotic Cook (AI Agent) parses the recipe exhaustively down to the milligram. The robot cook will never look up from the stove to ask if you prefer extra garlic; it executes the written text with uncompromising literalism.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Klasik yazılım mühendisliğinde kullanım senaryolarının yalnızca iki okuyucusu vardı: Doğrulayan iş birimi (business) ve kodu yazan insan geliştirici (developer). Cockburn'ün en önemli kuralı 'okunabilirlik' (readability) idi: 'Okunabilirliği feda ederseniz paydaşlarınız use case okumayı bırakır.' AI Unified Process üçüncü bir okuyucuyu sahneye çıkardı: AI Kodlama Ajanı. Ancak yapılan en ölümcül hata, 'Madem AI okuyor, öyleyse use case'i prompt mühendisliğine çevirelim, JSON veya pseudo-code yazalım' yanılgısıdır. Bir use case pseudo-code haline getirilirse paydaş onu okumadan imzalar; okunmadan imzalanan bir şartname ise kontrat olmaktan çıkar! Çözüm: İnsan için okunabilir olan kuralların (etken çatı, net özneler, tekil adımlar) aynı zamanda AI için de en az belirsizlik içeren format olduğunu anlamaktır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Historically, use cases served a dual audience: the business stakeholder who verified functional intent, and the software engineer who transcribed it into code. Cockburn made readability the supreme heuristic: an unreadable spec is abandoned by its business sponsors. The AI Unified Process introduces a third constituent: the autonomous AI agent. A catastrophic misconception is attempting to optimize the specification format exclusively for the machine via pseudo-code or prompt-engineering templates. Doing so alienates stakeholders, who sign off unread, turning the contract into fiction. Crucially, syntactic clarity (active voice, explicit subjects, named validation rules) satisfies both human skimming and machine parsing without compromise.

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: Üç Okuyucunun Ortak Dili Olan 'Clean Use Case Interactor'
package com.maang.aiup.readers;

public class CancelSubscriptionUseCase {
    // 1. Paydaşın anladığı iş modeli (Gözlemlenebilir Niyet)
    public record CancellationRequest(String memberId, String reason, boolean immediate) {}
    public record CancellationSummary(String memberId, boolean isTerminated, String refundStatus) {}

    // 2. Mühendisin denetlediği mimari kontrat
    public interface SubscriptionRepository {
        boolean hasActiveSubscription(String memberId);
        void recordCancellation(String memberId, String reason);
    }

    // 3. AI'ın doğrudan ürettiği ve test ettiği icra sınıfı
    public static class CancelSubscriptionInteractor {
        private final SubscriptionRepository repository;

        public CancelSubscriptionInteractor(SubscriptionRepository repository) {
            this.repository = repository;
        }

        public CancellationSummary execute(CancellationRequest request) {
            // Step 2: System validates active subscription exists (BR-014)
            if (!repository.hasActiveSubscription(request.memberId())) {
                throw new IllegalStateException("BR-014: İptal edilecek aktif abonelik bulunamadı.");
            }
            // Step 3: System records cancellation and calculates refund
            repository.recordCancellation(request.memberId(), request.reason());
            String refund = request.immediate() ? "REFUND_PRORATED" :
"NO_REFUND_ACTIVE_UNTIL_CYCLE_END";

            return new CancellationSummary(request.memberId(), true, refund);
        }
    }

    void main() {
        System.out.println("Üç okuyucunun niyetini taşıyan Clean Use Case hazır.");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Use case'i yapay zekaya hitap edeceğim diye Python/Java benzeri sözde kodlarla (pseudo-code) yazmayın! Bir dökümanı paydaş (iş analisti, ürün yöneticisi) okuyup gözüyle doğrulayamıyorsa, o döküman bir şartname değil teknik çöplüktür.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never degrade use cases into pseudo-code or machine-syntax prompts. If a business stakeholder cannot skim the narrative and validate whether it solves their problem, the contract is severed and the governance model collapses.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Use case'in 3 okuyucusu vardır: Paydaş doğrular, Mühendis inceler, AI ise doğrudan kod ve test üretir.

• Okunabilirlik her zaman ilk şarttır; makine için insanı devre dışı bırakan döküman şartname vasfını yitirir.

• İyi yazılmış bir senaryo hem meşgul bir paydaşı hem de harfi harfine çalışan AI'ı aynı anda tatmin eder.

🎯 Key Architectural Takeaways

• A use case has three readers: Stakeholder validates, Engineer reviews, AI generates.

• Readability remains paramount; a spec that humans cannot validate is no longer a contract.

• A disciplined, active-voice spec satisfies both the busy human stakeholder and the literal AI agent.

8

BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

Lafı Harfi Harfine Anlayan Okuyucu: AI Neden Bir Boşluğu Asla Doldurmadan Bırakamaz?

The Literal Reader: Why an AI Cannot Leave a Gap Unfilled

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic

```
┌──────────────────────────────────────────────────┐
│          BOŞLUK DOLDURMA MEKANİZMASI: İNSAN VS YAPAY ZEKÂ          │
└──────────────────────────────────────────────────┘

SENARYO: "Sistem başarısız giriş denemelerini yönetir." (Eksik / Muğlak Kural)

1. İNSAN MÜHENDİS:
   - Durur, düşünür.
   - "3 deneme mi 5 deneme mi? Kaç dakika kilitlenecek?" diye soru sorar.
   - BİLMEDİĞİNİN FARKINDADIR (Aware of Ignorance).

2. AI AJANI (LLM / Claude Code / Copilot):
   - Asla durmaz! Boşluğu hemen doldurur!
   - Kendi kafasından uydurur: "5 deneme olsun, 15 dakika kilitlensin, SMS atsın!"
   - TAM ÖZGÜVENLE SUNAR (Confident Hallucination).
   - Sonuç: Müşterinin hiç istemediği uydurma bir özellik artık canlı kodunuzda!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ressama ‘Bana bir oturma odası çiz’ deyip koltuğun rengini söylemezsen, ressam durup ‘Koltuk ne renk olsun?’ diye sorar. Ama yapay zekâ fırçayı eline alır almaz boş gördüğü her yere pembe puantiyeli koltuklar, tavana disko topu ve köşeye bir zürafa çizer! Neden? Çünkü yapay zekâ boş tuval görünce dayanamaz, istatistiksel olarak ne denk gelirse doldurur. Boşluk bırakırsan, o boşluğu senin yerine AI’ın zevkleri doldurur!

🇬🇧 ENGLISH (ELI5)

Ask an interior designer to draw a living room without specifying the sofa color, and they pause to ask your preference. Generative AI cannot tolerate an empty canvas: it instantly paints neon-leopard sofas, hangs a disco ball, and places a giraffe in the corner with total conviction. AI has no capacity to say ‘I notice a gap here, what did you mean?’ It completes the statistical distribution. Any intent you omit becomes an invented feature.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Yapay zekanın olasılıksal doğası (Autoregressive Token Generation) ve niyet boşlukları: LLM'ler bir sonraki en olası token'ı tahmin etmek üzere eğitilmiştir. Bu mimari zorunluluk gereği bir LLM 'Burada bir iş kuralı eksik, durup bekleyeyim' diyemez (bunu ancak harici meta-prompting veya /spec-review gibi deterministik araçlarla sağlayabilirsiniz). Use case içinde bir sınır, sıralama veya hata mesajı belirtilmemişse, model eğitim verisindeki binlerce açık kaynak projeden öğrendiği ortalama davranışı uygular. Örneğin: 'Kullanıcı şifresini unuttuysa sistem şifreyi sıfırlar.' AI burada parola sıfırlama linkinin süresini 24 saat mi yapacak, 15 dakika mı? Tek kullanımlık token mı üretecek? Eğer şartname 'BR-021: Parola sıfırlama token'ı 15 dakika geçerlidir' demezse, AI her kod üretiminde farklı bir süre uydurur. Yeniden üretilebilirlik (reproducibility) yok olur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Autoregressive generation dynamics and the impossibility of omission: LLMs are mathematical prediction engines optimized to generate plausible continuations. By structural design, an LLM cannot autonomously halt on a requirements vacuum and say 'The author omitted a timeout parameter; I shall suspend output.' Instead, it samples from its latent training distribution, selecting standard web defaults. If a use case states 'System blocks abusive IPs', the model will arbitrarily invent an arbitrary threshold (e.g. 10 requests per minute vs 100). The downstream consequence is devastating: the system exhibits behavioral drift across regenerations because implicit rules fluctuate with each prompt run.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: AI Boşluklarını Kapatan Tip Güvenli ve Parametrik Güvenlik Kuralı
package com.maang.aiup.literal;

import java.time.Duration;

public class LoginPolicyEngine {
    // Şartnameden birebir gelen ve AI'a hiçbir uydurma payı bırakmayan kural record'u
    public record LockoutPolicy(
        int maxFailedAttempts,        // BR-008: Kesin sayı (örn: 3)
        Duration lockoutDuration,      // BR-008: Kesin süre (örn: 15 dakika)
        boolean notifyAdminByEmail    // BR-009: Kesin bayrak
    ) {}

    public static final LockoutPolicy PRODUCTION_POLICY =
        new LockoutPolicy(3, Duration.ofMinutes(15), true);

    public record LoginAttemptResult(boolean allowed, String statusMessage) {}

    public static LoginAttemptResult evaluate(int consecutiveFails, LockoutPolicy policy) {
        if (consecutiveFails >= policy.maxFailedAttempts()) {
            return new LoginAttemptResult(false,
                "BR-008: Hesap " + policy.lockoutDuration().toMinutes() + " dakika süreyle
kilitlendi.");
        }
        return new LoginAttemptResult(true, "Giriş denemesi geçerli.");
    }

    void main() {
        var result = evaluate(3, PRODUCTION_POLICY);
        System.out.println("Deterministik sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir use case yazarken 'vb.', 've benzeri', 'gerekirse', 'uygun şekilde' kelimelerini kullanmak, yapay zekaya açık çek vermektir. Bu kelimeleri gördüğünüz yerde şartnameyi silip somut sınırları yazın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Using words like 'etc.', 'and so on', 'as appropriate', or 'if necessary' is writing a blank check to an AI agent. Every ambiguous clause is an architectural decision delegated to an algorithm.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yapay zekâ bir boşluğu asla boş bırakamaz; en olası gördüğü varsayımla sessizce doldurur.
- Yazılmayan gereksinim soru olarak dönmez; hatalı üretilmiş bir özellik olarak canlıya sızar.
- Tüm limitler, süreler ve hata akışları şartnamede eksiksiz ve somut olarak belirtilmelidir.

🎯 Key Architectural Takeaways

- An AI agent cannot leave a gap unfilled; it fills every omission with plausible invention.
- Unwritten requirements return as hallucinated features, not polite clarifying questions.
- All thresholds, durations, and failure paths must be explicitly bound within the specification.

9

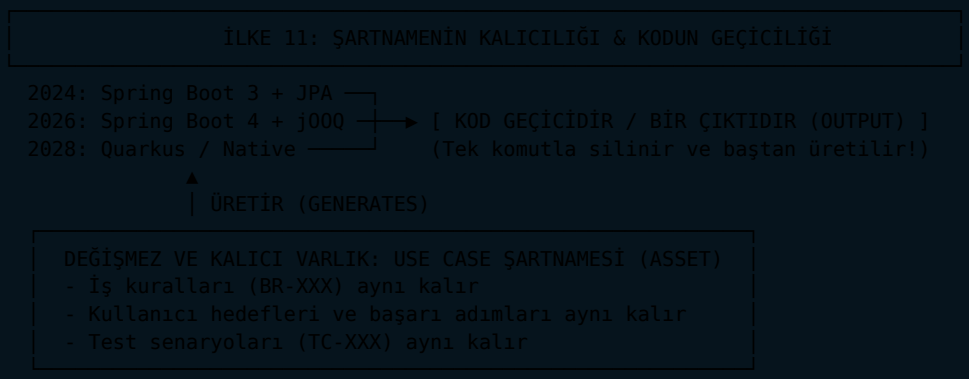
BÖLÜM: ÜÇ OKUYUCU MODELİ & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESİ | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 11: Şartname Koddan Daha Uzun Yaşar (The Spec Outlives the Code)

AI-Era Principle 11: The Specification Outlives the Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tiyatro oyununu düşünün: Hamlet oyunu 400 yıldır aynı senaryoyla oynanıyor. Ancak aktörler, kostümler, sahne ışıkları ve tiyatro binası yüzlerce kez değişti! Kod dediğin şey sahnedeki kostümdür; eskiyince çöpe atar yenisini giydirirsin. Use Case şartnamesi ise Shakespeare'in yazdığı ana tiyatro metnidir (Hamlet)! Kostüm değişir ama metin asla ölmez!

🇬🇧 ENGLISH (ELI5)

Consider Shakespeare's Hamlet: the play has endured for 400 years, while the actors, costumes, stages, and lighting rigs have been replaced thousands of times. Source code is merely the temporary stage costume—it wears out, gets refactored, and is ultimately discarded. The use case specification is the timeless script. Frameworks and syntax will fade, but the core business contract outlives them all.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Yazılım Mühendisliği 2.0 ve İlke 11'in Derinliği: Martinelli'nin Use-Case 3.0'a eklediği 11. ilke: 'Şartname koddan daha uzun yaşar.' Geleneksel yazılım geliştirmede kod en değerli varlıktı (asset); şartname ise geliştirme bitince bir köşeye atılan geçici bir iskeleydi (scaffolding). AI çağında bu durum 180 derece değişti. Bugün modern bir AI ajanı, elinde kaliteli bir use case ve veri modeli varsa, 10.000 satırlık bir mikroservisi 10 dakikada sıfırdan üretebilir. Ancak o 10.000 satırlık koda bakarak şirketin gerçek iş kurallarını (business intent) geri çıkarmak aylar sürer ve devasa hatalara gebedir. Bu nedenle bir projede kod ve şartname çeliştiğinde, varsayılan kural şudur: 'Şartname doğrudur, kod hatalıdır.' Kod her zaman şartnameye uydurulmak üzere yeniden üretilir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 11 Architectural Mandate: 'The specification outlives the code.' In legacy paradigms, code was the ultimate store of organizational value; specifications were discarded scaffolding. In modern AI-augmented software engineering, implementation is fungible. Armed with a rigorous use case model and entity dictionary, an LLM agent generates an entire production-grade subsystem in minutes. Conversely, reverse-engineering unstated business invariants from legacy binary bytecodes is agonizing and prone to catastrophic data loss. Therefore, whenever code and specification disagree, the architectural presumption is uncompromising: the code is defective. Code is transient; intent is permanent.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 10 Yıl Sonra Bile Geçerliliğini Koruyan Değişmez İş Kuralı Modeli
package com.maang.aiup.principles;

public class Principle11Demo {
    // 2026'da Java 25 ile, 2030'da Java 30 ile derlense de bu kural DEĞİŞMEZ:
    // BR-005: Bir üye aynı anda en fazla 5 açık ödünç kitaba sahip olabilir.
    public static final int MAX_OPEN_LOANS_LIMIT = 5;

    public record MemberLoanProfile(String memberId, int activeLoansCount) {
        public boolean canBorrowMore() {
            return activeLoansCount < MAX_OPEN_LOANS_LIMIT;
        }
    }

    void main() {
        var profile = new MemberLoanProfile("M-77", 4);
        System.out.printf("Üye ID: %s | Yeni kitap alabilir mi? %s (Limit: %d)%n",
            profile.memberId(), profile.canBorrowMore(), MAX_OPEN_LOANS_LIMIT);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir kodlama hatasını düzeltmek için doğrudan koda girip 'if (count < 6)' yazmayın! Bu satırı gören biri kuralın 5 mi 6 mı olduğunu anlayamaz. Önce şartnamedeki BR-005 kuralını güncelleyin, ardından kodu yeniden ürettirin.

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never patch an operational defect by directly tweaking conditional logic in source files without touching the specification. Unsynchronized code patches cause specification drift, guaranteeing that future automated rebuilds will overwrite your changes.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Kod bir derleme çıktısıdır; atılabilir, değiştirilebilir ve yeniden üretilebilir.

• Use Case şartnamesi kurumun ana entelektüel sermayesidir (asset) ve sistem yaşadıkça korunur.

• Kod ile şartname çeliştiğinde her zaman kod hatalı kabul edilir.

🎯 Key Architectural Takeaways

• Code is now a build artifact: disposable, replaceable, and effortlessly regenerated.

• The use case specification is the enterprise asset, maintained for the life of the system.

• When specification and code disagree, the default axiom is that the code is wrong.

10

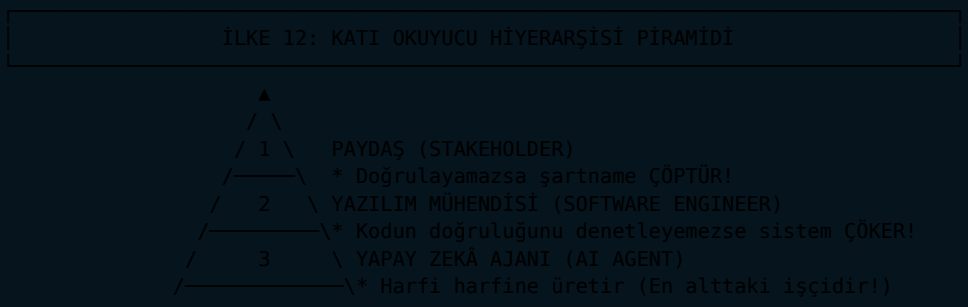
BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 12: Okuyucu Hiyerarşisi (Paydaş > Mühendis > Yapay Zekâ)

AI-Era Principle 12: Reader Hierarchy (Stakeholder > Engineer > AI)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette patron (Paydaş), başmimar (Mühendis) ve inşaat robotu (AI) vardır. Şirketin yeni binasının planı çizilirken kime göre çizilir? Elbette patrona ve mimara göre! Eğer robot daha rahat anlasın diye mimari planı '010101' şeklinde ikili bilgisayar kodlarıyla yazarsan, patron 'Ben bu binayı istemedim ki!' diyemez bile. İlk önce patron anlayacak, sonra mimar onaylayacak, en son robot tuğlayı koyacak!

🇬🇧 ENGLISH (ELI5)

In a corporate hierarchy, you have the Executive (Stakeholder), the Chief Architect (Engineer), and the Construction Drone (AI Agent). If you draft building blueprints in raw assembly bytecode so the drone can ingest them faster, the executive cannot verify whether the building has enough emergency exits. The blueprint must first be legible to the owner, verifiable by the engineer, and only then parsed by the drone.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 12: Üç okuyucu için şu katı öncelik sırasıyla yaz: 1) Paydaş, 2) Mühendis, 3) Yapay Zekâ. Martinelli'nin uyarısı: Eğer bir paydaş şartnameyi okuyup doğrulayamıyorsa, o şartnamenin taşıdığı teknik hassasiyet tamamen değersizdir. Eğer bir mühendis o şartnameye bakarak üretilen kodun doğruluğunu teyit edemiyorsa, sistemin güvenliği sıfırdır. Ancak ilk iki okuyucu tatmin edildikten sonra yapay zekanın tüketebilirliği (AI-consumability) önem kazanır. Harika olan gerçek şudur: İlk iki okuyucuya mükemmel hizmet eden bir use case (etken çatı, açık aktör isimleri, 'kontrol eder' yerine 'doğrular', numaralı adımlar), neredeyse otomatik olarak üçüncü okuyucuya (AI) da kusursuz hizmet eder. Bu yüzden okuyucular arasında bir ödünleşim (trade-off) hissettiğinizde, neredeyse kesinlikle gereksiz bir uygulama detayına (implementation mechanism) sapmışsınız demektir!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 12 Governance Formula: 'Write for three readers, in strict descending order: Stakeholder, Engineer, AI.' If a business stakeholder cannot comprehend the specification, its algorithmic precision is worthless. If a software engineer cannot cross-examine the generated code against the spec, correctness cannot be validated. Only after these human gates are established does machine consumability matter. Fortunately, these needs converge: active voice, single-responsibility steps, and explicit business rules satisfy humans and machines simultaneously. If you ever feel compelled to sacrifice human readability for AI parsing, you have invariably leaked implementation mechanisms into the specification.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Paydaşın, Mühendisin ve AI'ın Birlikte Doğrulayabildiği Domain Modeli
package com.maang.aiup.hierarchy;

public class ReaderHierarchyDemo {
    // Paydaş Okur: "Üye, Kurs ve Kayıt" kelimelerini görür.
    // Mühendis Okur: Tip güvenliğini ve değişmezliği (immutability) görür.
    // AI Okur: Hangi validation kurallarının işletileceğini eksiksiz anlar.
    public record CourseEnrollment(
        String enrollmentId,
        String memberId,
        String courseCode,
        boolean isPrerequisiteSatisfied,
        int availableSeats
    ) {
        public CourseEnrollment {
            // Step 4: System validates prerequisites (BR-020)
            if (!isPrerequisiteSatisfied) {
                throw new IllegalStateException("BR-020: Önkoşul dersleri tamamlanmamış.");
            }
            // Step 5: System validates seat availability (BR-021)
            if (availableSeats <= 0) {
                throw new IllegalStateException("BR-021: Kursta boş kontenjan kalmadı.");
            }
        }
    }

    void main() {
        var enrollment = new CourseEnrollment("ENR-10", "MEM-5", "CS-101", true, 15);
        System.out.println("✅ Üç okuyucunun da onayladığı kayıt: " + enrollment);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameye JSON şemaları, SQL sorguları veya regex desenleri koyarak 'AI böyle daha iyi anlıyor' bahanesine sığınmayın. Bu tür detaylar paydaşın okumasını engeller ve İlke 12'yi doğrudan ihlal eder.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never inline regex formulas, SQL statements, or JSON schemas into use case steps under the pretext that 'the AI understands code better'. Doing so alienates stakeholders and destroys the foundational reader hierarchy.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Okuyucu sırası asla değişmez: 1. Paydaş, 2. Mühendis, 3. Yapay Zekâ.
- Paydaşın onaylamadığı şartnameyle üretilen kod, ne kadar hatasız çalışırsa çalışsın çöptür.
- İnsanın kolay okuduğu yapı (aktif dil, numaralı adımlar), AI için de en az yanılsama üreten yapıdır.

🎯 Key Architectural Takeaways

- Reader priority is immutable: 1. Stakeholder, 2. Software Engineer, 3. AI Agent.
- Code generated from an unvalidated spec is useless, regardless of algorithmic elegance.
- Structures that maximize human readability also minimize AI hallucination rates.

11

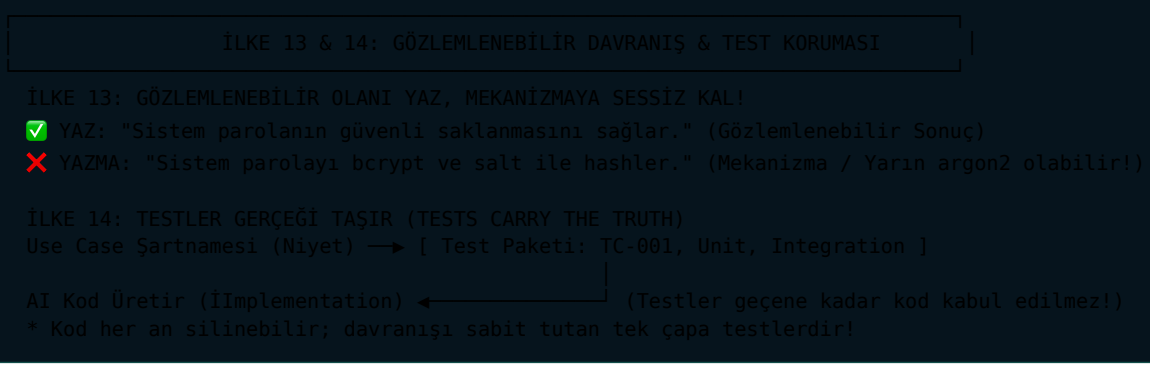
BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkeleri 13 & 14: Gözlemlenebilir Davranış vs Testler Gerçeği Taşır

AI-Era Principles 13 & 14: Observable Behavior vs Tests Carry the Truth

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İlke 13: Bankaya gittiğinde veznedara 'Bana paramı ver ama parayı verirken sol elini 45 derece açıyla kaldır, kasayı anahtarla 3 kere çevir' demezsin. Sadece '1000 TL çekmek istiyorum' dersin (Sonuç!). Kasayı nasıl açtığı bankanın iç meselesidir. İlke 14: Veznedarın parayı doğru verip vermediğini anlamak için parayı eline alır sayarsın (Test!). Parayı saymazsan eksik mi verdi sahte mi verdi bilemezsin!

🇬🇧 ENGLISH (ELI5)

Principle 13: At an ATM, you request '$100 cash'. You never instruct the machine: 'Spin gear 4 at 120 RPM and trigger hydraulic valve B'. The mechanical implementation is the machine's private affair. Principle 14: When the bills emerge, you count them. Tests are the physical counting of the cash. A use case without automated tests is merely wishful thinking.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 13 ve 14'ün Sistem Mimarisi Açısından Önemi: 1) İlke 13: Gözlemlenebilir davranış (observable behavior) ile mekanizma (mechanism) arasındaki ayrım, bir şartnamenin ömrünü belirler. Bir kullanıcının veya testin dışarıdan gözlemleyebileceği her şey (ekrandaki kolonlar, hata mesajları, 28 günlük vade gibi kesin sayılar) Zone 1'dir ve yazılmalıdır. Sistemin bunu nasıl başardığı (SQL query, JWT token, bcrypt, Redis cache) Zone 3'tür ve KESİNLİKLE YASAKTIR. Mekanizma sızarsa şartname mimariyi kilitler ve framework yükseltmelerinde çöker. 2) İlke 14: 'Testler gerçeği taşır' (Tests carry the truth). Use-Case 3.0'ın en çarpıcı tespiti: Test senaryoları bir use case'in anlatısından bile daha önemlidir. Çünkü AI çağında kodu yeniden üretmek (regeneration) ancak davranış dışarıdan bir test takımıyla sabitlenmişse güvenlidir. Kod değişir, silinir, baştan yazılır; ancak testler yeşil kaldığı sürece sistemin sözleşmesi korunmuş demektir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Deep dive on Principles 13 & 14: 1) Principle 13 (Observable vs Mechanism): Replaces decades of ambiguous advice regarding 'the right level of specification detail'. If a stakeholder can verify it or an automated test can assert it, it belongs in the specification (Zone 1). If it describes internal computational wiring (protocols, database schemas, hashing algorithms), it belongs to the implementation skill (Zone 3). Leaking Zone 3 mechanics guarantees that the spec will contradict future framework upgrades. 2) Principle 14 (Tests Carry the Truth): In an automated regeneration lifecycle, source code is volatile. What keeps the system's behavior invariant across LLM rewrites is the test suite. Every use case flow must yield an executable test case; without tests, automated code generation is reckless gambling.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Mekanizmadan Bağımsız Davranışsal Test (Tests Carry the Truth)
package com.maang.aiup.principles;

public class ObservableBehaviorTest {
    // Sözleşme: Parola formatı doğrulanır (Mekanizma: regex mi değil mi şartnameye yazılmaz!)
    public interface PasswordValidator {
        boolean isValid(String password);
    }

    // İlke 14: Gerçeği taşıyan test paketi
    public static void runTests(PasswordValidator validator) {
        // Test 1: En az 8 karakter olmalı
        assert !validator.isValid("short") : "8 karakterden kısa parola reddedilmeli!";
        // Test 2: Rakam içermeli
        assert !validator.isValid("NoDigitsHere!") : "Rakam içermeyen parola reddedilmeli!";
        // Test 3: Başarılı parola
        assert validator.isValid("SecureP@ss2026") : "Geçerli parola kabul edilmeli!";

        System.out.println("✅ İlke 14: Tüm davranışsal testler başarıyla geçti!");
    }

    void main() {
        // AI'ın ürettiği implementasyon (İster regex kullansın, ister döngü!)
        PasswordValidator aiGenerated = pwd -> pwd != null && pwd.length() >= 8 &&
pwd.matches(".*\d.*");
        runTests(aiGenerated);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Şartnameye 'Parola bcrypt ile hashlenir' yazarsanız, yarın kuantum güvenli veya Argon2 algoritmasına geçtiğinizde şartname yalan söylemeye başlar. 'Parola güvenli şekilde saklanır' yazın; algoritma seçimini mimari yapılandırmaya bırakın.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

If you write 'password is hashed with bcrypt' in a use case, the moment your team upgrades to Argon2id, the specification becomes a lie. Write 'system securely stores password' and delegate cryptography to the tech stack configuration.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Paydaşın görebildiği ve testin doğrulayabildiği her şey yazılır; mekanizma asla yazılmaz.
- Teknolojiler, kütüphaneler ve veri tabanı komutları şartnameye sızarsa sistem kilitlenir.
- Testler olmadan AI ile kod üretmek kumar oynamaktır; gerçeği kod değil testler taşır.

🎯 Key Architectural Takeaways

- Specify everything observable and testable; stay completely silent on internal mechanisms.
- Leaking protocols, SQL, and algorithms into specs makes future modernization impossible.
- Without automated tests, AI code generation is blind; tests carry the enduring truth.

12

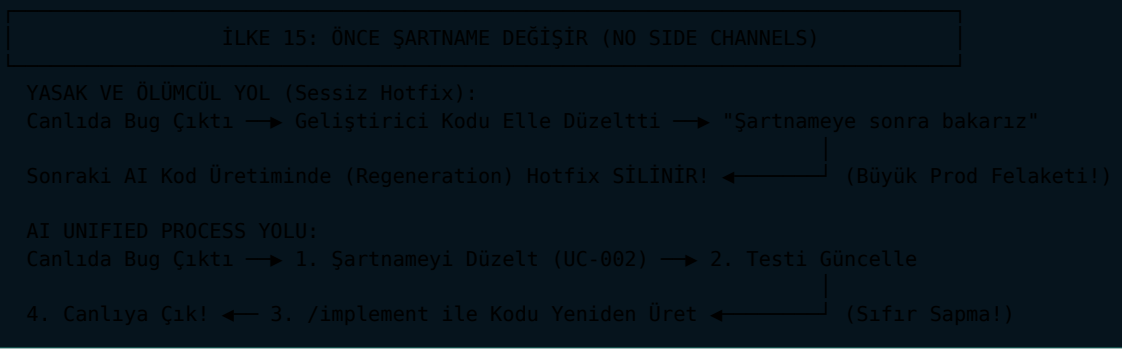
BÖLÜM: ÜÇ OKUYUCU MODELI & YAPAY ZEKÂ ÇAĞININ 5 TEMEL İLKESI | MODULE: THE THREE READERS & THE 5 AI-ERA PRINCIPLES

AI-Çağı İlkesi 15: Önce Şartname Değişir (Sessiz Hotfix Kültürüne Son!)

AI-Era Principle 15: Change the Specification First (Killing the Silent Hotfix)

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir geminin rotasını değiştirmek istediğinde dümencinin eline gizlice vurup dümeni çevirmezsin. Önce kaptan köşküne gider haritayı güncellersin! Eğer haritayı güncellemeden dümene asılırsan, bir sonraki vardiyada gelen yeni dümenci (AI) haritaya bakar ve gemiyi tekrar eski rotaya kırıp kayalıklara çarpar! Hata çıksa da yeni özellik gelse de önce harita (şartname) değişir!

🇬🇧 ENGLISH (ELI5)

You don't alter a ship's heading by sneaking into the wheelhouse and wrestling the steering wheel away from the helmsman. You update the navigational nautical chart. If you yank the wheel without recording the new course on the chart, the next watchman (the AI generator) inspects the chart, swings the rudder back, and rams the vessel into an iceberg. The map changes first, always.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

İlke 15: Önce şartnameyi değiştir (Change the specification first). Yeni bir özellik (feature), bir değişiklik talebi (change request) ve bir hata (bug)... AI Unified Process'te bu üç kavramın hepsi TEK VE AYNI ŞEYDİR: 'Bir use case'e yapılan değişiklik.' Sistemde /bug veya /hotfix adında özel bir komut yoktur! Hata çıktığında süreç şöyledir: 1) Hatanın niyetini sınıflandır (Kod mu saptı yoksa şartname mi yanlıştı?). 2) Şartnameyi güncelle. 3) /implement komutunu yeniden çalıştır. AI mevcut sınıfları bulur ve şartnamedeki değişikliğe göre yerinde günceller (in-place reconciliation). Eğer acil durum bahanesiyle kodu doğrudan IDE'den düzeltip şartnameyi güncellemezseniz, şartnamenin 'gerçeklik kaynağı' olma vasfı anında ölür. Bir sonraki sprint'te birisi `/implement` çalıştırdığında elle yazdığınız o hotfix tamamen silinir!

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Principle 15 Operational Discipline: 'Change the specification first.' A new feature, a change request, and a production bug report are fundamentally identical abstractions: a delta applied to a use case specification. The AI Unified Process provides no special `/hotfix` or `/patch` command. The protocol is singular: 1) Classify intent (did code diverge from spec, or was the spec flawed?). 2) Update the use case markdown file. 3) Re-invoke `/implement`. The generative engine reconciles existing classes in-place against the updated specification. Bypassing this loop under production pressure instantly corrupts the artifact chain. The next automated regeneration cycle will treat your undocumented hotfix as dead code and purge it.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Şartname Değişikliğinin Koddaki Yansıması (Regeneration Friendly)
package com.maang.aiup.principles;

public class LoanInterestCalculation {
    // ESKİ KURAL BR-012: Faiz oranı sabit %5.0 idi.
    // YENİ KURAL BR-012 (Değişiklik Şartnamede Yapıldı): VIP üyeler için faiz %3.5, diğerleri %5.0.

    public record Customer(String id, boolean isVip) {}

    public static double calculateInterestRate(Customer customer) {
        // Şartnamedeki BR-012 güncellemesinden sonra AI tarafından üretilen yeni kod:
        if (customer.isVip()) {
            return 3.5; // BR-012 (Revize edilmiş şartname kuralı)
        }
        return 5.0;     // BR-012 Standart kural
    }

    void main() {
        var vip = new Customer("VIP-1", true);
        var standard = new Customer("STD-2", false);

        System.out.println("VIP Faiz: %" + calculateInterestRate(vip));
        System.out.println("Standart Faiz: %" + calculateInterestRate(standard));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Geliştiricilerin 'Hata çok acil, prod yanıyor, önce kodu pushlayalım şartnameyi pazartesi yazarız' tuzağına ASLA düşmeyin. O pazartesi hiçbir zaman gelmez. Şartnameyi değiştirmek 30 saniye, /implement çalıştırmak 10 saniyedir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Beware the fatal team rationalization: 'Production is down, let us hot-patch the code now and backfill the markdown spec on Monday.' That Monday never arrives. Updating the spec takes 30 seconds; letting it rot costs weeks of technical bankruptcy.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Yeni özellik, değişiklik ve bug: Hepsi aslında use case şartnamesine yapılan bir değişikliktir.
- Kodu şartnameden geçirmeden doğrudan düzeltmek yasaktır; yan kanallar şartnameyi çürütür.
- Şartname güncellendiğinde aynı /implement ve test komutları çalışarak kodu yerinde senkronize eder.

🎯 Key Architectural Takeaways

- Features, change requests, and bugs are all the exact same thing: a delta to a use case.
- Bypassing the specification via direct code hotfixes triggers immediate specification rot.
- When the spec changes, running the same commands reconciles existing code in place cleanly.